



C o m m u n i t y E x p e r i e n c e D i s t i l l e d

Mastering Concurrency in Go

Discover and harness Go's powerful concurrency features to develop and build fast, scalable network systems

Nathan Kozyra

[PACKT] open source*
PUBLISHING community experience distilled

Mastering Concurrency in Go

Discover and harness Go's powerful concurrency features to develop and build fast, scalable network systems

Nathan Kozyra



Mastering Concurrency in Go

Copyright © 2014 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: July 2014

Production reference: 1160714

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-78398-348-3

www.packtpub.com

Cover image by Pratyush Mohanta (tysoncinematics@gmail.com)

Credits

Author

Nathan Kozyra

Project Coordinator

Danuta Jones

Reviewers

Jeremy R. Budnack

János Fehér

Aleksandar S. Sokolovski

Michele Della Torre

Proofreaders

Simran Bhogal

Ameesha Green

Paul Hindle

Commissioning Editor

Julian Ursell

Indexers

Hemangini Bari

Priya Subramani

Acquisition Editor

Kevin Colaco

Graphics

Abhinash Sahu

Content Development Editor

Athira Laji

Production Coordinator

Conidon Miranda

Technical Editors

Pratik More

Pooja Nair

Shweta S. Pant

Rohit Kumar Singh

Cover Work

Conidon Miranda

Copy Editors

Roshni Banerjee

Sarang Chari

Janbal Dharmaraj

Sayanee Mukherjee

Karuna Narayanan

Laxmi Subramanian

About the Author

Nathan Kozyra has been programming both recreationally and professionally for more than a decade now. His first experience came while writing games on his grandfather's Commodore 64, and in the ensuing decades, he's crafted technological solutions and applications in nearly every major language for a host of software and media companies as a developer, advisor, creative technologist, and CTO. He is currently the CTO of Pointslocal. A new-language enthusiast and C++ stalwart, his attention was quickly captured by Google's Go language, both for the language's creators and ethos as well as its apparent post-C approach to systems languages. Having dived in quickly, Go is now his go-to language for fast, concurrent applications.

This book is dedicated to Mary and Ethan.

About the Reviewers

Jeremy R. Budnack is a software engineer and consultant from Buffalo, NY. His career includes almost 15 years of experience in software development and systems administration, spanning many diverse technology stacks and programming languages (including C#, Ruby, and Go). He currently works for Stark & Wayne, LLC., specializing in building cloud-based solutions using a variety of technologies including Cloud Foundry, BOSH, OpenStack, and vSphere.

János Fehér has been involved in a wide variety of projects since 1996, including technical support for NATO operations and development of high-performance computing grids, national TV and radio websites, and web applications for universities and adult learning. In recent years, he has been heavily involved in distributed and concurrent software architectures. He is currently the Head of Development for Intern Avenue.

To my love, Szilvi, who painted my cycling shoes full with gophers.

Aleksandar Stefan Sokolovski is a project manager and an ICT research engineer from Skopje, Macedonia. He has a Bachelor of Science (BSc) degree in Computer Science from the Ss. Cyril and Methodius University and a Master of Science (MSc) degree in Technology, Innovation, and Entrepreneurship from the University of Sheffield, U.K. He has been a member of an organization committee (CITYR'09 / CITYR'11), a participant (CIIT'11), and a presenter (XIX Scientific Conference IT 2014) at multiple international research conferences; he is also a published author of many research papers for international conferences including a published paper in a journal (ETF Journal of Electrical Engineering, 2014). He has worked as a research associate and teaching assistant at the Faculty for Informatics and Computer Science in Skopje, Macedonia. He currently works as a research associate at the Institute for Digital Forensics and as an IT engineer, a research engineer, and a R&D project coordinator at telecommunications and Internet provider Neotel, Skopje from Republic of Macedonia. He is a member of the Executive Committee of IEEE R8, Section in Republic of Macedonia as Professional Activities Officer and is also a member of Project Management Institute (PMI)

I am heartily thankful to my mentors, professors Dragan Mihajlov, Toni, and Saso Gelev, whose encouragement, guidance, and support from the initial to the final level enabled me to develop an understanding and the knowledge to take on a significant part of this project. I offer my regards and respect to all my colleagues for their support and all their help. I would like to thank Packt Publishing and its team working on the book.

Lastly, I offer my regards and blessings to all of those who supported me in any respect during the completion of the project: my family for their support and all their help, especially my sister Sofija, my parents Stefan and Brankica, and last but not least, my girlfriend Kalina.

Michele Della Torre is a software engineer, programmer, and geek born in Italy in 1982. In love with computers since he was a child, he started studying computer science at high school and obtained his Master's degree in Computer Engineering in 2007 at the Politecnico of Milan university. After university, he continued to learn about software design and architectures, agile methodologies, concurrent systems, and algorithms. He currently works in a home automation company and is occasionally involved in open source projects.

I'd like to thank my parents, my girlfriend Alessia, and my friends Alex and Fabio for their continuous support and patience.

www.PacktPub.com

Support files, eBooks, discount offers, and more

You might want to visit www.PacktPub.com for support files and downloads related to your book.

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.PacktPub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at service@packtpub.com for more details.

At www.PacktPub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters and receive exclusive discounts and offers on Packt books and eBooks.



<http://PacktLib.PacktPub.com>

Do you need instant solutions to your IT questions? PacktLib is Packt's online digital book library. Here, you can access, read and search across Packt's entire library of books.

Why subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print and bookmark content
- On demand and accessible via web browser

Free access for Packt account holders

If you have an account with Packt at www.PacktPub.com, you can use this to access PacktLib today and view nine entirely free books. Simply use your login credentials for immediate access.

Table of Contents

Preface	1
Chapter 1: An Introduction to Concurrency in Go	9
Introducing goroutines	9
A patient goroutine	11
Implementing the defer control mechanism	13
Using Go's scheduler	15
Using system variables	20
Understanding goroutines versus coroutines	21
Implementing channels	22
Channel-based sorting at the letter capitalization factory	25
Cleaning up our goroutines	29
Buffered or unbuffered channels	29
Using the select statement	29
Closures and goroutines	33
Building a web spider using goroutines and channels	35
Summary	40
Chapter 2: Understanding the Concurrency Model	41
Understanding the working of goroutines	41
Synchronous versus asynchronous goroutines	42
Designing the web server plan	42
Visualizing concurrency	44
RSS in action	49
An RSS reader with self diagnostics	50
Imposing a timeout	57
A little bit about CSP	57
The dining philosophers problem	58
Go and the actor model	60

Object orientation	60
Demonstrating simple polymorphism in Go	61
Using concurrency	62
Managing threads	62
Using sync and mutexes to lock data	63
Summary	65
Chapter 3: Developing a Concurrent Strategy	67
Applying efficiency in complex concurrency	67
Identifying race conditions with race detection	68
Using mutual exclusions	72
Exploring timeouts	77
Importance of consistency	78
Synchronizing our concurrent operations	78
The project – multiuser appointment calendar	79
Visualizing a concurrent pattern	81
Developing our server requirements	82
Web server	82
Using templates	83
Time	84
Endpoints	84
Custom structs	85
A multiuser Appointments Calendar	85
A note on style	93
A note on immutability	94
Summary	95
Chapter 4: Data Integrity in an Application	97
Getting deeper with mutexes and sync	97
The cost of goroutines	100
Working with files	101
Getting low – implementing C	103
Touching memory in cgo	104
The structure of cgo	104
The other way around	105
Getting even lower – assembly in Go	109
Distributed Go	111
Some common consistency models	113
Distributed shared memory	113
First-in-first-out – PRAM	114
Looking at the master-slave model	114
The producer-consumer problem	115
Looking at the leader-follower model	116

Atomic consistency / mutual exclusion	117
Release consistency	117
Using memcached	118
Circuit	119
Summary	121
Chapter 5: Locks, Blocks, and Better Channels	123
Understanding blocking methods in Go	124
Blocking method 1 – a listening, waiting channel	124
Sending more data types via channels	125
The net package – a chat server with interfaced channels	130
Examining our client	137
Blocking method 2 – the select statement in a loop	139
Cleaning up goroutines	140
Blocking method 3 – network connections and reads	141
Creating channels of channels	141
Pprof – yet another awesome tool	143
Handling deadlocks and errors	150
Summary	151
Chapter 6: C10K – A Non-blocking Web Server in Go	153
Attacking the C10K problem	154
Failing of servers at 10,000 concurrent connections	155
Using concurrency to attack C10K	156
Taking another approach	158
Building our C10K web server	159
Benchmarking against a blocking web server	160
Handling requests	162
Routing requests	165
Serving pages	166
Parsing our template	167
External dependencies	171
Connecting to MySQL	171
Multithreading and leveraging multiple cores	174
Exploring our web server	174
Timing out and moving on	178
Summary	180
Chapter 7: Performance and Scalability	181
High performance in Go	182
Getting deeper into pprof	182
Parallelism's and concurrency's impact on I/O pprof	188
Using the App Engine	191

Distributed Go	192
Types of topologies	192
Type 1 – star	193
Type 2 – mesh	196
The Publish and Subscribe model	197
Serialized data	198
Remote code execution	198
Other topologies	199
Message Passing Interface	200
Some helpful libraries	201
Nitro profiler	201
Heka	202
GoFlow	202
Memory preservation	202
Garbage collection in Go	203
Summary	203
Chapter 8: Concurrent Application Architecture	205
Designing our concurrent application	206
Identifying our requirements	207
Using NoSQL as a data store in Go	208
MongoDB	209
Redis	211
Tiedot	213
CouchDB	214
Cassandra	215
Couchbase	216
Setting up our data store	219
Monitoring filesystem changes	221
Managing logfiles	222
Handling configuration files	223
Detecting file changes	224
Sending changes to clients	226
Checking records against Couchbase	229
Backing up our files	231
Designing our web interface	233
Reverting a file's history – command line	238
Using Go in daemons and as a service	240
Checking the health of our server	241
Summary	242

Chapter 9: Logging and Testing Concurrency in Go	243
Handling errors and logging	244
Breaking out goroutine logs	246
Using the LiteIDE for richer and easier debugging	248
Sending errors to screen	249
Logging errors to file	250
Logging errors to memory	252
Using the log4go package for robust logging	254
Panicking	259
Recovering	260
Logging our panics	263
Catching stack traces with concurrent code	265
Using the runtime package for granular stack traces	265
Summary	269
Chapter 10: Advanced Concurrency and Best Practices	271
Going beyond the basics with channels	272
Building workers	272
Implementing nil channel blocks	278
Using nil channels	280
Implementing more granular control over goroutines with tomb	281
Timing out with channels	284
Building a load balancer with concurrent patterns	286
Choosing unidirectional and bidirectional channels	292
Using receive-only or send-only channels	293
Using an indeterminate channel type	293
Using Go with unit testing	295
GoCheck	296
Ginkgo and Gomega	296
Using Google App Engine	297
Utilizing best practices	297
Structuring your code	298
Documenting your code	298
Making your code available via go get	299
Keeping concurrency out of your packages	299
Summary	300
Index	301

Preface

I just love new programming languages. Perhaps it's the inevitable familiarity and ennui with regard to existing languages and the frustration with existing tools, syntaxes, coding conventions, and performance. Maybe I'm just hunting for that one "language to rule them all". Whatever the reason, any time a new or experimental language is released, I have to dive right in.

This has been a golden age for new languages and language design. Think about it: the C language was released in the early 1970s — a time when resources were so scarce that verbosity, clarity, and syntactical logic were often eschewed for thrift. And most of the languages we use today were either originally written in this era or were directly influenced by those languages.

Since the late 1980s and early 1990s, there has been a slow flood of powerful new languages and paradigms — Perl, Python, Ruby, PHP, and JavaScript — have taken an expanding user base by storm and has become one of the most popular languages (up there with stalwarts such as C, C++, and Java). Multithreading, memory caching, and APIs have allowed multiple processes, dissonant languages, applications, and even separate operating systems to work in congress.

And while this is great, there's a niche that until very recently was largely unserved: powerful, compiled, cross-platform languages with concurrency support that are geared towards systems programmers.

Very few languages match these parameters. Sure, there have been lower-level languages that fulfill some of these characteristics. Erlang and Haskell fit the bill in terms of power and language design, but as functional languages they pose a learning barrier for systems programmers coming from a C/Java background. Objective-C and C# are relatively easy, powerful, and have concurrency support—but they're bound enough to a specific OS to make programming for other platforms arduous. The languages we just mentioned (Python, JavaScript, and so on)—while extremely popular—are largely interpreted languages, forcing performance into a secondary role. You can use most of them for systems programming, but in many ways it's the proverbial square peg in a round hole. So when Google announced Go in 2009, my interest was piqued. When I saw who was behind the project (more on that later), I was elated. When I saw the language and its design in action, I was in heaven.

For the last few years I've been using Go to replace systems applications I'd previously written in C, Java, Perl, and Python. I couldn't be happier with the results. Implementing Go has improved these applications in almost every instance. The fact that it plays nicely with C is another huge selling point for systems programmers looking to dip their toes in Go's pool.

With some of the best minds in language design (and programming in general) behind it, Go has a bright future.

For years—decades, really—there have been less than a handful of options for writing servers and network interfaces. If you were tasked with writing one, you probably reached for C, C++, or Java. And while these certainly can handle the task, and while they all now support concurrency and parallelism in some way or another, they weren't designed for that.

Google brought together a team that included some giants of programming—Rob Pike and Ken Thompson of Bell Labs fame and Robert Griesemer, who worked on Google's JavaScript implementation V8—to design a modern, concurrent language with development ease at the forefront.

To do this, the team focused on some sore spots in the alternatives, which are as follows:

- Dynamically typed languages have—in recent years—become incredibly popular. Go eschews the explicit, "cumbersome" type systems of Java or C++. Go uses type inference, which saves development time, but is still also strongly typed.
- Concurrency, parallelism, pointers/memory access, and garbage collection are unwieldy in the aforementioned languages. Go lets these concepts be as easy or as complicated as you want or need them to be.

- As a newer language, Go has a focus on multicore design that was a necessary afterthought in languages such as C++.
- Go's compiler is super-fast; it's so fast that there are implementations of it that treat Go code as interpreted.
- Although Google designed Go to be a systems language, it's versatile enough to be used in a myriad of ways. Certainly, the focus on advanced, cheap concurrency makes it ideal for network and systems programming.
- Go is loose with syntax, but strict with usage. By this we mean that Go will let you get a little lazy with some lexer tokens, but you still have to produce fundamentally tight code. As Go provides a formatting tool that attempts to clarify your code, you can also spend less time on readability concerns as you're coding.

What this book covers

Chapter 1, An Introduction to Concurrency in Go, introduces goroutines and channels, and will compare the way Go handles concurrency with the approach other languages use. We'll build some basic concurrent applications utilizing these new concepts.

Chapter 2, Understanding the Concurrency Model, focuses on resource allocation, sharing memory (and when not to), and data. We will look at channels and channels of channels as well as explain exactly how Go manages concurrency internally.

Chapter 3, Developing a Concurrent Strategy, discusses approach methods for designing applications to best use concurrent tools in Go. We'll look at some available third-party packages that can play a role in your strategy.

Chapter 4, Data Integrity in an Application, looks at ensuring that delegation of goroutines and channels maintain the state in single thread and multithread applications.

Chapter 5, Locks, Blocks, and Better Channels, looks at how Go can avoid dead locks out of the box, and when and where they can still occur despite Go's language design.

Chapter 6, C10K – A Non-blocking Web Server in Go, tackles one of the Internet's most famous and esteemed challenges and attempt to solve it with core Go packages. We'll then refine the product and test it against common benchmarking tools.

Chapter 7, Performance and Scalability, focuses on squeezing the most out of your concurrent Go code, best utilizing resources and accounting for and mitigating third-party software's impact on your own. We'll add some additional functionality to our web server and talk about other ways in which we can use these packages.

Chapter 8, Concurrent Application Architecture, focuses on when and where to implement concurrent patterns, when and how to utilize parallelism to take advantage of advanced hardware, and how to ensure data consistency.

Chapter 9, Logging and Testing Concurrency in Go, focuses on OS-specific methods for testing and deploying your application. We'll also look at Go's relationship with various code repositories.

Chapter 10, Advanced Concurrency and Best Practices, looks at more complicated and advanced techniques including duplicating concurrent features not available in Go's core.

What you need for this book

To work along with this book's examples, you'll need a computer running Windows, OS X, or quite a few Linux variants that support Go. For this book, our Linux examples and notes reference Ubuntu.

If you do not already have Go 1.3 or newer installed, you will need to get it either from the binaries download page on <http://golang.org/> or through your operating system's package manager.

To use all of the examples in this book, you'll also need to have the following software installed:

- MySQL (<http://dev.mysql.com/downloads/>)
- Couchbase (<http://www.couchbase.com/download>)

Your choice of IDE is a matter of personal preference, as anyone who's worked with developers can attest. That said, there are a few that lend themselves better to some languages than others and a couple that have good support for Go. This author uses Sublime Text, which plays very nice with Go, is lightweight, and allows you to build directly from within the IDE itself. Anywhere you see screenshots of code, it will be from within Sublime Text.

And while there's a good amount of baked-in support for Go code, there's also a nice plugin collection for Sublime Text called GoSublime, available at <https://github.com/DisposaBoy/GoSublime>.

Sublime Text isn't free, but there is a free evaluation version available that has no time limit. It's available in Windows, OS X, and Linux variants at <http://www.sublimetext.com/>.

Who this book is for

If you are a systems or network programmer with some knowledge of Go and concurrency, but would like to know about the implementation of concurrent systems written in Go this is the book for you. The goal of this book is to enable you to write high-performance, scalable, resource-thrifty systems and network applications in Go.

In this book, we'll write a number of basic and somewhat less - basic network and systems applications. It's assumed that you've worked with these types of applications before. If you haven't, some extracurricular study may be warranted to be able to fully digest this content.

Conventions

In this book, you will find a number of styles of text that distinguish between different kinds of information. Here are some examples of these styles, and an explanation of their meaning.

Code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles are shown as follows: "The `setProxy` function is called after every request, and you can see it as the first line in our handler."

A block of code is set as follows:

```
package main

import (
    "net/http"
    "html/template"
    "time"
    "regexp"
    "fmt"
    "io/ioutil"
    "database/sql"
    "log"
    "runtime"
    _ "github.com/go-sql-driver/mysql"
)
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
package main

import (
    "fmt"
)

func stringReturn(text string) string {
    return text
}

func main() {
    myText := stringReturn("Here be the code")
    fmt.Println(myText)
}
```

Any command-line input or output is written as follows:

```
go get github.com/go-sql-driver/mysql
```

New terms and **important words** are shown in bold. Words that you see on the screen, in menus or dialog boxes for example, appear in the text like this: "If you upload a file by dragging it to the **Drop files here to upload** box, within a few seconds you'll see that the file is noted as changed in the web interface."



Warnings or important notes appear in a box like this.



Tips and tricks appear like this.

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book – what you liked or may have disliked. Reader feedback is important for us to develop titles that you really get the most out of.

To send us general feedback, simply send an e-mail to feedback@packtpub.com, and mention the book title via the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide on www.packtpub.com/authors.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at <http://www.packtpub.com>. If you purchased this book elsewhere, you can visit <http://www.packtpub.com/support> and register to have the files e-mailed directly to you.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books – maybe a mistake in the text or the code – we would be grateful if you would report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting <http://www.packtpub.com/submit-errata>, selecting your book, clicking on the **errata submission form** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded on our website, or added to any list of existing errata, under the Errata section of that title. Any existing errata can be viewed by selecting your title from <http://www.packtpub.com/support>.

Piracy

Piracy of copyright material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at copyright@packtpub.com with a link to the suspected pirated material.

We appreciate your help in protecting our authors, and our ability to bring you valuable content.

Questions

You can contact us at questions@packtpub.com if you are having a problem with any aspect of the book, and we will do our best to address it.

1

An Introduction to Concurrency in Go

While Go is both a great general purpose and low-level systems language, one of its primary strengths is the built-in concurrency model and tools. Many other languages have third-party libraries (or extensions), but inherent concurrency is something unique to modern languages, and it is a core feature of Go's design.

Though there's no doubt that Go excels at concurrency – as we'll see in this book – what it has that many other languages lack is a robust set of tools to test and build concurrent, parallel, and distributed code.

Enough talk about Go's marvelous concurrency features and tools, let's jump in.

Introducing goroutines

The primary method of handling concurrency is through a goroutine. Admittedly, our first piece of concurrent code (mentioned in the preface) didn't do a whole lot, simply spitting out alternating "hello"s and "world"s until the entire task was complete.

Here is that code once again:

```
package main

import (
    "fmt"
    "time"
)

type Job struct {
    i int
    max int
    text string
}

func outputText(j *Job) {
    for j.i < j.max {
        time.Sleep(1 * time.Millisecond)
        fmt.Println(j.text)
        j.i++
    }
}

func main() {
    hello := new(Job)
    world := new(Job)

    hello.text = "hello"
    hello.i = 0
    hello.max = 3

    world.text = "world"
    world.i = 0
    world.max = 5

    go outputText(hello)
    outputText(world)
}
```



Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at <http://www.packtpub.com>. If you purchased this book elsewhere, you can visit <http://www.packtpub.com/support> and register to have the files e-mailed directly to you.

But, if you think back to our real-world example of planning a surprise party for your grandmother, that's exactly how things often have to be managed with limited or finite resources. This asynchronous behavior is critical for some applications to run smoothly, although our example essentially ran in a vacuum.

You may have noticed one quirk in our early example: despite the fact that we called the `outputText()` function on the `hello` struct first, our output started with the `world` struct's text value. Why is that?

Being asynchronous, when a goroutine is invoked, it waits for the blocking code to complete before concurrency begins. You can test this by replacing the `outputText()` function call on the `world` struct with a goroutine, as shown in the following code:

```
go outputText(hello)
go outputText(world)
```

If you run this, you will get no output because the main function ends while the asynchronous goroutines are running. There are a couple of ways to stop this to see the output before the main function finishes execution and the program exits. The classic method simply asks for user input before execution, allowing you to directly control when the application finishes. You can also put an infinite loop at the end of your main function, as follows:

```
for {}
```

Better yet, Go also has a built-in mechanism for this, which is the `WaitGroup` type in the `sync` package.

If you add a `WaitGroup` struct to your code, it can delay execution of the main function until after all goroutines are complete. In simple terms, it lets you set a number of required iterations to get a completed response from the goroutines before allowing the application to continue. Let's look at a minor modification to our "Hello World" application in the following section.

A patient goroutine

From here, we'll implement a `WaitGroup` struct to ensure our goroutines run entirely before moving on with our application. In this case, when we say patient, it's in contrast to the way we've seen goroutines run outside of a parent method with our previous example. In the following code, we will implement our first `waitgroup` struct:

```
package main

import (
    "fmt"
    "sync"
    "time"
)
```

```
type Job struct {
    i int
    max int
    text string
}

func outputText(j *Job, goGroup *sync.WaitGroup) {
    for j.i < j.max {
        time.Sleep(1 * time.Millisecond)
        fmt.Println(j.text)
        j.i++
    }
    goGroup.Done()
}

func main() {

    goGroup := new(sync.WaitGroup)
    fmt.Println("Starting")

    hello := new(Job)
    hello.text = "hello"
    hello.i = 0
    hello.max = 2

    world := new(Job)
    world.text = "world"
    world.i = 0
    world.max = 2

    go outputText(hello, goGroup)
    go outputText(world, goGroup)

    goGroup.Add(2)
    goGroup.Wait()

}
```

Let's look at the changes in the following code:

```
goGroup := new(sync.WaitGroup)
```

Here, we declared a `WaitGroup` struct named `goGroup`. This variable will receive note that our goroutine function has completed x number of times before allowing the program to exit. Here's an example of sending such an expectation in `WaitGroup`:

```
goGroup.Add(2)
```

The `Add()` method specifies how many `Done` messages `goGroup` should receive before satisfying its wait. Here, we specified 2 because we have two functions running asynchronously. If you had three goroutine members and still called two, you may see the output of the third. If you added a value more than two to `goGroup`, for example, `goGroup.Add(3)`, then `WaitGroup` would wait forever and deadlock.

With that in mind, you shouldn't manually set the number of goroutines that need to wait; this is ideally handled computationally or explicitly in a range. This is how we tell `WaitGroup` to wait:

```
goGroup.Wait()
```

Now, we wait. This code will fail for the same reason `goGroup.Add(3)` failed; the `goGroup` struct never receives messages that our goroutines are done. So, let's do this as shown in the following code snippet:

```
func outputText(j *Job, goGroup *sync.WaitGroup) {
    for j.i < j.max {
        time.Sleep(1 * time.Millisecond)
        fmt.Println(j.text)
        j.i++
    }
    goGroup.Done()
}
```

We've only made two changes to our `outputText()` function from the preface. First, we added a pointer to our `goGroup` as the second function argument. Then, when all our iterations were complete, we told `goGroup` that they are all done.

Implementing the defer control mechanism

While we're here, we should take a moment and talk about `defer`. Go has an elegant implementation of the defer control mechanism. If you've used `defer` (or something functionally similar) in other languages, this will seem familiar—it's a useful way of delaying the execution of a statement until the rest of the function is complete.

For the most part, this is syntactical sugar that allows you to see related operations together, even though they won't execute together. If you've ever written something similar to the following pseudocode, you'll know what I mean:

```
x = file.open('test.txt')
int longFunction() {
...
}
x.close();
```

You probably know the kind of pain that can come from large "distances" separating related bits of code. In Go, you can actually write the code similar to the following:

```
package main

import(
    "os"
)

func main() {

    file, _ := os.Create("/defer.txt")

    defer file.Close()

    for {

        break

    }

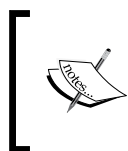
}
```

There isn't any actual functional advantage to this other than making clearer, more readable code, but that's a pretty big plus in itself. Deferred calls are executed reverse of the order in which they are defined, or last-in-first-out. You should also take note that any data passed by reference may be in an unexpected state.

For example, refer to the following code snippet:

```
func main() {  
  
    aValue := new(int)  
  
    defer fmt.Println(*aValue)  
  
    for i := 0; i < 100; i++ {  
        *aValue++  
    }  
  
}
```

This will return 0, and not 100, as it is the default value for an integer.



Defer is not the same as *deferred* (or futures/promises) in other languages. We'll talk about Go's implementations and alternatives to futures and promises in *Chapter 2, Understanding the Concurrency Model*.

Using Go's scheduler

With a lot of concurrent and parallel applications in other languages, the management of both soft and hard threads is handled at the operating system level. This is known to be inherently inefficient and expensive as the OS is responsible for context switching, among multiple processes. When an application or process can manage its own threads and scheduling, it results in faster runtime. The threads granted to our application and Go's scheduler have fewer OS attributes that need to be considered in context to switching, resulting in less overhead.

If you think about it, this is self-evident—the more you have to juggle, the slower it is to manage all of the balls. Go removes the natural inefficiency of this mechanism by using its own scheduler.

There's really only one quirk to this, one that you'll learn very early on: if you don't ever yield to the main thread, your goroutines will perform in unexpected ways (or won't perform at all).

Another way to look at this is to think that a goroutine must be blocked before concurrency is valid and can begin. Let's modify our example and include some file I/O to log to demonstrate this quirk, as shown in the following code:

```
package main

import (
    "fmt"
    "time"
    "io/ioutil"
)

type Job struct {
    i int
    max int
    text string
}

func outputText(j *Job) {
    fileName := j.text + ".txt"
    fileContents := ""
    for j.i < j.max {
        time.Sleep(1 * time.Millisecond)
        fileContents += j.text
        fmt.Println(j.text)
        j.i++
    }
    err := ioutil.WriteFile(fileName, []byte(fileContents), 0644)
    if (err != nil) {
        panic("Something went awry")
    }
}

func main() {

    hello := new(Job)
    hello.text = "hello"
    hello.i = 0
    hello.max = 3
}
```

```
world := new(Job)
world.text = "world"
world.i = 0
world.max = 5

go outputText(hello)
go outputText(world)

}
```

In theory, all that has changed is that we're now using a file operation to log each operation to a distinct file (in this case, `hello.txt` and `world.txt`). However, if you run this, no files are created.

In our last example, we used a `sync.WaitSync` struct to force the main thread to delay execution until asynchronous tasks were complete. While this works (and elegantly), it doesn't really explain *why* our asynchronous tasks fail. As mentioned before, you can also utilize blocking code to prevent the main thread from completing before its asynchronous tasks.

Since the Go scheduler manages context switching, each goroutine must yield control back to the main thread to schedule all of these asynchronous tasks. There are two ways to do this manually. One method, and probably the ideal one, is the `WaitGroup` struct. Another is the `GoSched()` function in the `runtime` package.

The `GoSched()` function temporarily yields the processor and then returns to the current goroutine. Consider the following code as an example:

```
package main

import (
    "runtime"
    "fmt"
)

func showNumber(num int) {
    fmt.Println(num)
}

func main() {
    iterations := 10

    for i := 0; i<=iterations; i++ {
```

```
        go showNumber(i)

    }
    //runtime.Gosched()
    fmt.Println("Goodbye!")
}
```

Run this with `runtime.Gosched()` commented out and the underscore before `"runtime"` removed, and you'll see only `Goodbye!`. This is because there's no guarantee as to how many goroutines, if any, will complete before the end of the `main()` function.

As we learned earlier, you can explicitly wait for a finite set number of goroutines before ending the execution of the application. However, `Gosched()` allows (in most cases) for the same basic functionality. Remove the comment before `runtime.Gosched()`, and you should get 0 through 10 printed before `Goodbye!`.

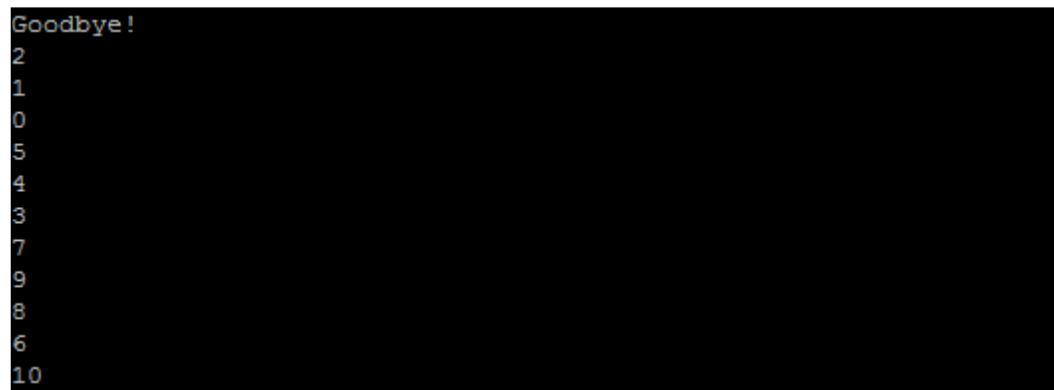
Just for fun, try running this code on a multicore server and modify your max processors using `runtime.GOMAXPROCS()`, as follows:

```
func main() {

    runtime.GOMAXPROCS(2)
```

Also, push your `runtime.Gosched()` to the absolute end so that all goroutines have a chance to run before `main` ends.

Got something unexpected? That's not unexpected! You may end up with a totally jostled execution of your goroutines, as shown in the following screenshot:



```
Goodbye!
2
1
0
5
4
3
7
9
8
6
10
```